SHIV NADAR
INSTITUTION OF EMINENCE DEEMED TO BE
UNIVERSITY
DELHI NCR

NH-91, Tehsil Dadri
Gautam Buddha Nagar
Uttar Pradesh - 201314, India

Tel: +91-120-7170100
+91-120-2662002

Date: 28 Nov 2025

Certificate

This is to certify that the project titled **"Unsupervised Image Segmentation and Denoising"** is submitted by **Shourey agarwal** (Roll no.2210110957), **Aryaman Srivastava** (Roll no.2210110206) and **Adeep Sharma** (Roll no.2210110116),Computer Science and Engineering, Shiv Nadar IOE in the partial fulfilment of the requirements for the award of the degree of **Bachelor of Technology in Computer Science and Engineering**. This work was completed under the supervision of Dr. Saurabh Shigwan. No part of this dissertation/report has been submitted elsewhere for award of any other degree.

Signature with date: ______________ 29/11/25

Name of Supervisor: Dr. Saurabh Shigwan

Designation: Assistant Professor

Affiliation: Shiv Nadar Institution of Eminence

Unsupervised Image Segmentation and Denoising

Project Report Submitted in Partial Fulfilment of the Requirements for the Degree of

Bachelor of Technology

in

Computer Science and Engineering

Submitted by

Adeep Sharma: 2210110116
Aryaman Srivastava: 2210110206
Shourey Agarwal: 2210110957

Under the Supervision of
Dr. Saurabh Shigwan
Assistant Professor

SHIV NADAR
INSTITUTION OF EMINENCE DEEMED TO BE
UNIVERSITY
DELHI NCR

Department of Computer Science and Engineering

Chapter 1: Introduction

1.1 Background

Image segmentation and denoising are fundamental tasks in computer vision, with segmentation partitioning an image into meaningful regions based on color, texture or intensity; and denoising removing unwanted noise while preserving important structures. Traditional supervised methods for these tasks require large labeled datasets which are often expensive or impractical to obtain, making unsupervised learning a valuable alternative that learns patterns directly from raw data. Combining segmentation and denoising is particularly effective, as noise can hinder segmentation accuracy, while segmentation can help preserve edges and structural details during denoising. These techniques have wide-ranging real-world applications, including medical imaging (X-rays, MRIs, CT scans), satellite imagery and remote sensing, autonomous systems like self-driving vehicles, industrial quality inspection, surveillance and astronomical imaging. Overall, unsupervised joint segmentation and denoising provide robust label-free solutions critical for practical image analysis in complex and noisy environments.

1.2 Motivation

The motivation for this project arises from the limitations of traditional supervised learning approaches in real-world image segmentation and denoising. Supervised methods rely on large-scale labeled datasets, which are expensive, time-consuming and often impractical to obtain, especially for tasks requiring pixellevel annotations or paired noisy and clean images. Manual labeling is labor-intensive, prone to inconsistencies, and may not generalize across different domains or imaging conditions leading to degraded performance under domain shifts. Additionally, supervised models struggle in noisy environments, as they require clean reference images that are frequently unavailable and are brittle when handling unanticipated noise. Unsupervised learning offers a promising solution by discovering meaningful patterns without labels and overcoming human bias. It provides scalability for large datasets, flexibility in diverse applications and the potential to uncover insights that humans might overlook. Developing effective unsupervised segmentation and denoising techniques addresses both practical challenges and the broader goal of creating AI systems capable of learning autonomously from raw data.

1.3 Problem Statement

The objective of this project is to develop an unsupervised learning model capable of performing **image segmentation and denoising simultaneously**. Given a noisy input image without ground truth labels or clean references, the model must produce a **segmented image** that partitions the image into meaningful regions and a **denoised image** that removes noise while preserving important structural details. The challenges include identifying semantic regions without supervision, effectively removing noise without blurring edges or textures generalizing across diverse image types and noise characteristics and designing a framework where segmentation and denoising improve each other. Additional considerations include computational efficiency for practical deployment. Success will be evaluated both **quantitatively** and **qualitatively** through visual inspection, ensuring meaningful segmentation and natural-looking denoised outputs. The goal is a **robust, scalable system** that handles real-world imaging tasks without requiring manual annotation, advancing unsupervised learning in computer vision.

1.4 Objectives

The primary objectives of this project are designed to tackle the key challenges identified in the problem statement and to advance the field of unsupervised learning in computer vision. They provide a clear roadmap for the research, implementation, and evaluation throughout the project.

Objective 1: Design an Unsupervised Model for Joint Segmentation and Denoising
Develop a neural network capable of simultaneously performing **image segmentation and denoising** in a fully unsupervised manner. The model should include a feature extraction backbone, a segmentation module for clustering pixels into meaningful regions and a denoising module that preserves structural details. The components should **interact**, with segmentation guiding denoising and vice versa using carefully designed unsupervised loss functions to learn meaningful patterns without labeled data.

Objective 2: Develop a Scalable and Generalizable Pipeline
Build a complete pipeline that is **computationally efficient, scalable to large datasets and generalizable across diverse imaging domains and conditions**. This includes preprocessing steps, efficient training procedures, modular design for adaptability and architectural choices that promote learning robust, transferable features. The pipeline should perform well on multiple datasets, including natural images, medical scans and satellite imagery.

Objective 3: Evaluate Performance on Diverse Image Datasets
Establish a **rigorous evaluation framework** using quantitative metrics for segmentation and denoising alongside qualitative visual inspection. Test the model under varying **noise types, noise levels and image complexities**, including out-of-distribution datasets to validate generalization and ensure that the outputs are both mathematically sound and perceptually meaningful.

Objective 4: Compare with Existing Unsupervised Techniques
Perform a **systematic comparison** with existing unsupervised and self-supervised methods, including classical clustering, graph-based segmentation traditional denoising methods and modern self-supervised networks.

Chapter 2: Related Work

2.1 Image Segmentation Techniques

Image segmentation is one of the most fundamental problems in computer vision with a rich history spanning several decades. The field has evolved from simple thresholding methods to sophisticated deep learning approaches each offering different trade-offs between accuracy, computational efficiency and supervision requirements.

2.1.1 Traditional Segmentation Methods

Traditional image segmentation techniques rely on **handcrafted features and classical algorithms** that do not require labeled training data. **K-means clustering** is one of the earliest and most common methods grouping pixels into K clusters based on color similarity. Although simple and computationally efficient it performs poorly on complex images with varying illumination, textures or overlapping color distributions and requires prior knowledge of K. **Mean-shift segmentation** improves upon this by performing nonparametric clustering that automatically identifies the number of clusters but it is computationally expensive and sensitive to the bandwidth parameter. **Graph-based methods** such as *Normalized Cuts* and *Felzenszwalb's algorithm* represent images as graphs with pixels as nodes and edges denoting similarity enabling them to capture global image structure and produce perceptually coherent regions though at the cost of high computation and parameter sensitivity. **Watershed segmentation** which interprets image intensity as topography to identify catchment basins effectively separates touching objects and preserves boundaries but often suffers from over-segmentation necessitating careful preprocessing or post-processing.

2.1.2 Deep Learning Approaches

Deep learning has transformed image segmentation with **CNN-based models** achieving exceptional accuracy. **U-Net** originally designed for biomedical tasks became a foundational model due to its encoder decoder structure with skip connections that balance detail preservation and semantic understanding. **Fully Convolutional Networks (FCNs)** introduced end-to-end pixel-wise prediction. A**ttention mechanisms** and **Transformer-based models** enhanced performance by modeling long-range dependencies. However, these methods depend heavily on large annotated datasets prompting research into **unsupervised and self-supervised approaches** that reduce reliance on manual labeling.

2.2 Unsupervised Segmentation Methods

Unsupervised segmentation aims to partition images into meaningful regions without relying on labeled training data. This challenging problem has gained significant attention as researchers seek to develop more scalable and practical segmentation solutions.

2.2.1 Clustering-Based Deep Learning Approaches

Clustering-based deep learning approaches integrate feature learning with clustering to achieve unsupervised segmentation. Methods like **Deep Embedded Clustering (DEC)** jointly optimize feature representations and cluster assignments by iteratively updating network weights and cluster centers. This treats segmentation as a **pixel-wise clustering task** in a learned feature space, enabling the model to group similar regions without labeled data.

2.2.2 Self-Supervised and Contrastive Learning

Self-supervised learning enables representation learning without labeled data by comparing augmented views of the same image. Adapted for segmentation, methods like **PixelContrast** and **DenseCL** apply

contrastive learning at the pixel or region level, grouping similar pixels and separating dissimilar ones to discover semantic segments without explicit supervision.

2.2.3 Recent State-of-the-Art Methods

Recent state-of-the-art unsupervised segmentation methods achieve near-supervised performance using advanced learning strategies. **STEGO** combines vision transformers with contrastive learning to produce consistent high-quality segments. **PiCIE** uses instance-level contrastive learning with clustering to form invariant embeddings and discover semantic groupings. **IIC (Invariant Information Clustering)** maximizes mutual information between augmented image pairs to learn meaningful augmentation-invariant clusters.

2.2.4 TokenCut: Self-Supervised Graph Cuts for Segmentation

TokenCut is a recent breakthrough in unsupervised image segmentation, combining **self-supervised vision transformers** with **graph-based segmentation**. It uses features from **DINO-trained Vision Transformers (ViTs)** to build a similarity graph, where image patches are nodes and edges represent feature similarity. By applying **normalized graph cuts** on this graph, TokenCut identifies meaningful object boundaries without any labeled data.

Its innovation lies in leveraging the **semantic clustering ability of DINO features** enabling segmentation in a **training-free manner** during inference. The method extracts patch-level features, constructs a weighted graph and uses **spectral graph theory** to find natural partitions.

TokenCut achieves **strong results** on datasets like **ImageNet**, **PASCAL VOC** and **CUB-200-2011**, effectively discovering object parts such as bird heads, wings and tails **without supervision**. However, it faces limitations: it struggles with noisy images, lacks cross-image consistency and depends on the biases of pre-trained transformers, highlighting areas for future improvement.

2.2.5 EAGLE: Eigenvector-Guided Adaptive Learning for Efficient Segmentation

EAGLE represents a significant advancement in unsupervised semantic segmentation by combining **spectral clustering with deep feature learning**. Unlike TokenCut's training-free approach, EAGLE employs a trainable framework with a dual-probe architecture: a cluster probe that discovers natural groupings through K-means clustering, and a linear probe that maps these clusters to semantically consistent categories across images.

Building on DINO vision transformers, EAGLE extracts dense features and applies eigenvalue decomposition to identify principal components capturing dominant visual patterns. The linear probe ensures cross-image semantic consistency, a key advantage over pure clustering methods that assign arbitrary labels.

EAGLE achieves approximately **20-25% mIoU on COCO-Stuff-27**, demonstrating strong performance on standard benchmarks. However, it requires dataset-specific training and careful hyperparameter tuning, offering different trade-offs compared to TokenCut's training-free inference while providing superior semantic consistency.

2.3 Image Denoising Techniques

Image denoising is a classical problem in image processing that has seen continuous evolution from statistical methods to modern deep learning approaches. The goal is to estimate a clean image from its noisy observation while preserving important structures and details.

2.3.1 Classical Denoising Methods

Classical image denoising methods rely on mathematical and statistical models to reduce noise while preserving image details. **Gaussian filtering** smooths images by averaging nearby pixel values but tends to blur edges. **Non-Local Means (NLM)** improves upon this by averaging pixels with similar neighborhoods across the image preserving textures well but at a high computational cost. **Bilateral filtering** maintains edge sharpness by considering both spatial and intensity similarity though it can introduce artifacts in textured regions. **BM3D** (Block-Matching and 3D filtering) remains one of the best non-learning methods, grouping similar patches and filtering them collaboratively in the transform domain, though it is computationally heavy. **Wavelet-based denoising** uses multi-scale decomposition and coefficient thresholding to separate noise from signal, offering strong theoretical grounding but limited effectiveness on complex or non-Gaussian noise.

2.3.2 Deep Learning Denoising Methods

Deep learning has dramatically improved denoising performance through learned representations and endto-end optimization. Convolutional denoising autoencoders learn to map noisy images to clean images through encoder-decoder architectures. These networks can be trained on paired noisy-clean examples and achieve excellent performance when such training data is available.

DnCNN (Denoising Convolutional Neural Network) introduced residual learning for denoising, where the network predicts the noise component rather than the clean image directly. This formulation simplifies the learning task and improves performance. DnCNN and its variants have become standard baselines for supervised denoising.

However, obtaining paired training data remains challenging in many practical scenarios. This limitation has motivated the development of self-supervised denoising methods that do not require clean reference images.

2.3.3 Self-Supervised Denoising

Noise2Noise demonstrated the surprising result that neural networks can be trained to denoise images using only pairs of noisy images of the same scene without ever seeing clean images. The key insight is that the network learns to predict the expected clean image by averaging out noise across different noise realizations. While powerful, this approach still requires multiple noisy observations of the same scene which may not be available in many applications.

Noise2Void eliminates the need for multiple noisy observations by training a network to predict each pixel from its surrounding context excluding the pixel itself. This blind-spot network architecture ensures the network cannot simply learn an identity mapping and must instead learn to denoise by exploiting local image correlations. Noise2Void works on single noisy images without requiring paired or multiple observations, making it highly practical.

Noise2Self generalizes the blind-spot concept by partitioning pixels into disjoint sets and training the network to predict one set from the other. This theoretical framework provides a principled foundation for self-supervised denoising and has inspired various extensions.

These self-supervised methods represent significant advances in practical denoising, enabling effective noise removal without the traditional requirements of clean reference images. However, they typically focus solely on denoising without considering how segmentation information might improve performance.

2.4 Joint Segmentation and Denoising

While segmentation and denoising are often treated as independent problems, there is growing recognition that these tasks can benefit from joint modeling. The presence of noise complicates segmentation by obscuring boundaries and creating spurious features, while accurate segmentation can guide denoising by identifying regions with homogeneous properties.

2.4.1 Traditional Joint Methods

Classical approaches to joint segmentation and denoising often involve iterative optimization procedures that alternate between segmentation and denoising steps.

Active contours and level set methods have been extended to handle noisy images by incorporating denoising terms into the energy functional. These methods evolve segmentation boundaries to minimize energy that balances data fidelity, smoothness, and segmentation quality.

While theoretically elegant, these classical joint methods often struggle with initialization sensitivity, local minima and computational cost. They also require careful parameter tuning and may not generalize well across diverse image types and noise characteristics.

2.4.2 Deep Learning Joint Frameworks

Recent deep learning approaches have explored joint modeling of segmentation and denoising through multi-task learning frameworks. These methods typically employ shared encoder networks that extract common features useful for both tasks with separate decoder heads for producing segmentation maps and denoised images.

Some approaches use segmentation as a prior for denoising where predicted segment boundaries guide the application of different denoising strategies to different regions. Conversely, other methods first denoise the image and then perform segmentation on the cleaned result though this sequential approach may not fully leverage the synergy between tasks.

2.4.3 Unsupervised Joint Learning

Unsupervised joint segmentation and denoising remains relatively underexplored compared to its supervised counterpart. The challenge lies in designing learning objectives that encourage meaningful segmentation and effective denoising without access to ground truth for either task.

Some recent work has explored using reconstruction-based losses combined with clustering objectives where the network learns to segment images in a way that facilitates better denoising within each segment. Other approaches leverage consistency regularization encouraging the model to produce similar segmentations and denoising results for differently augmented versions of the same noisy image.

The TokenCut framework, while primarily designed for segmentation provides a potential foundation for joint modeling. By operating on features that are robust to noise (through self-supervised pre-training), TokenCut implicitly performs some noise handling. However, it does not explicitly optimize for denoising and may still be affected by severe noise that corrupts feature representations.

2.5 Gap Analysis and Motivation for Current Work

Despite significant progress in both unsupervised segmentation and self-supervised denoising, several important gaps remain in the literature that motivate the current project.

Limited Integration of Segmentation and Denoising: Most existing methods treat segmentation and denoising as separate problems, failing to exploit the natural synergy between these tasks. While some supervised methods have explored joint modeling, unsupervised joint frameworks remain underdeveloped.

Sensitivity to Noise in Unsupervised Segmentation: State-of-the-art unsupervised segmentation methods like TokenCut, EAGLE, STEGO and PiCIE typically assume relatively clean input images. When applied to noisy data, their performance degrades significantly as noise corrupts feature representations and introduces spurious patterns.

Lack of Comparative Analysis: While individual methods like EAGLE and TokenCut have been evaluated independently, comprehensive comparisons between different unsupervised segmentation approaches on the same datasets and evaluation protocols are limited. Understanding the relative strengths and trade-offs of these methods is crucial for practical applications.

Our Implementation Focus: This project addresses these gaps through:

1. **Comparative Implementation on COCO-Stuff**: We implement and compare both EAGLE and TokenCut on the COCO-Stuff-27 dataset, providing direct performance comparisons under identical conditions. This benchmark enables fair comparison as both methods report results on this dataset.
2. **Binary Segmentation Task**: We evaluate both methods on binary object segmentation (thing vs. stuff classes), a practical task that tests the ability to distinguish foreground objects from background regions.
3. **Implementation of TokenCut**: We implement TokenCut's full algorithm including:
    - GPU-accelerated processing for efficiency
    - Normalized Cut on DINO features
    - Proper evaluation on ECSSD and CUB-200-2011 datasets
4. **Optimization and Efficiency**: We optimize TokenCut's implementation for GPU acceleration, reducing processing time from hours to minutes (from ~7 seconds per image to ~0.15 seconds), making it practical for real-world deployment.
5. **Comprehensive Metrics**: We provide quantitative evaluation using binary mIoU metrics and qualitative visualization on COCO-Stuff, enabling thorough assessment of segmentation quality.

The COCO-Stuff dataset provides an excellent comparison benchmark because:

- Both EAGLE and TokenCut report results on this dataset
- It contains diverse scenes with 27 semantic categories (thing and stuff classes)
- Enables evaluation of binary segmentation (thing vs. stuff)
- Tests semantic consistency across varied object types and backgrounds

Additionally, we implement and evaluate TokenCut on ECSSD (salient object detection) and CUB-200-2011 (fine-grained bird segmentation) to demonstrate its applicability across different domains.

Building upon both EAGLE's learned semantic consistency and TokenCut's training-free graph-based approach, our comparative analysis on COCO-Stuff provides insights into when each method excels and their respective limitations, establishing a foundation for future work on joint segmentation and denoising.

2.6 Summary

This literature review has covered the evolution of image segmentation from classical methods to modern deep learning approaches, with particular emphasis on unsupervised and self-supervised techniques. We examined image denoising methods ranging from statistical approaches to recent self-supervised neural networks.

We highlighted two state-of-the-art unsupervised segmentation methods: **EAGLE**, which combines spectral clustering with learned semantic consistency through a dual-probe architecture and **TokenCut**, which leverages self-supervised vision transformers with graph-based segmentation in a training-free manner. Both methods build upon DINO features but offer different trade-offs between training requirements, semantic consistency and computational efficiency.

The review identified significant gaps in existing work, particularly the limited comparative analysis between different unsupervised segmentation approaches and their sensitivity to noisy inputs. These gaps motivate our project's focus on:

1. **Comparative evaluation of EAGLE and TokenCut** on COCO-Stuff-27 for binary segmentation (thing vs. stuff)
2. **Implementation and optimization of TokenCut** with GPU acceleration for efficient processing
3. **Evaluation of TokenCut on additional datasets** (ECSSD and CUB-200-2011) to demonstrate cross-domain applicability
4. **Comprehensive quantitative and qualitative analysis** to understand method strengths and limitations

Our work on the COCO-Stuff dataset establishes a direct comparison between EAGLE and TokenCut under identical conditions, while our TokenCut implementation on ECSSD and CUB-200-2011 demonstrates its effectiveness on salient object detection and fine-grained segmentation tasks. The following chapters detail our methodology, implementation optimizations, experimental setup, and comparative results focusing on the COCO-Stuff benchmark.

Chapter 3: TokenCut Implementation and Evaluation

3.1 Introduction

Image segmentation remains a fundamental challenge in computer vision, with applications ranging from medical imaging to autonomous driving. While supervised deep learning methods have achieved remarkable performance, they require large-scale annotated datasets that are expensive and time-consuming to create. This limitation has motivated the development of unsupervised segmentation methods that can discover and segment objects without manual annotations.

TokenCut represents a significant breakthrough in unsupervised image segmentation by combining self-supervised vision transformers with classical graph-based segmentation techniques. Unlike recent deep learning approaches that require extensive training on large datasets, TokenCut operates in a training-free manner at inference time, making it particularly attractive for practical applications where computational resources are limited or rapid deployment is required.

The method leverages rich semantic representations from **DINO (Self-Distillation with No Labels)**, a self-supervised vision transformer, and applies **Normalized Cut**—a classical graph partitioning algorithm—to segment images based on feature similarity. This elegant combination of modern self-supervised learning and classical spectral methods achieves competitive performance without task-specific training.

This chapter presents our implementation and evaluation of TokenCut on two datasets: **ECSSD** for salient object detection and **CUB-200-2011** for fine-grained bird segmentation. Our implementation serves as a strong baseline for comparison with learned approaches and provides insights into the capabilities and limitations of training-free unsupervised segmentation methods.

3.2 TokenCut Methodology

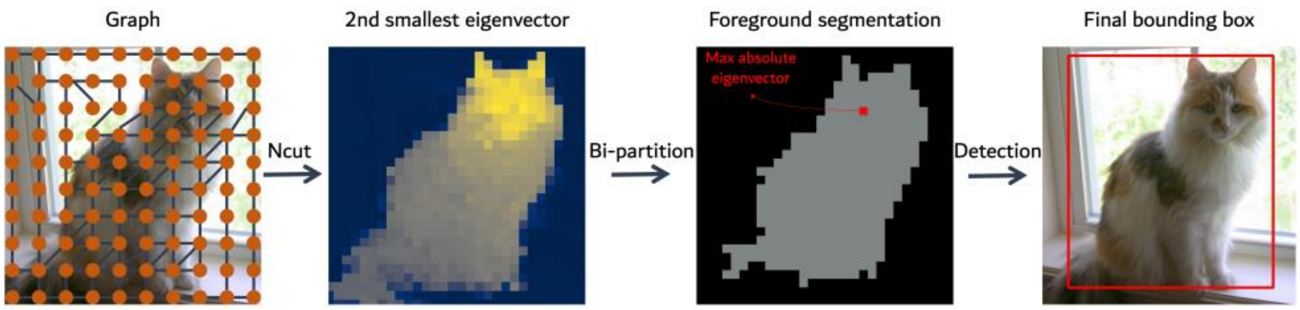


Figure: *TokenCut Architecture*

3.2.1 Algorithm Overview

TokenCut combines self-supervised vision transformers with graph-based segmentation to achieve training-free unsupervised object discovery. The method consists of three core components that work together to produce binary segmentation masks.

Self-Supervised Vision Transformers (DINO)

DINO (Self-Distillation with No Labels) is a self-supervised learning framework that trains Vision Transformers (ViTs) without manual annotations. Through a student-teacher distillation process with momentum updates and multi-crop augmentation, DINO learns rich semantic representations that naturally cluster similar image regions. The key property exploited by TokenCut is that DINO features exhibit strong semantic clustering - pixels belonging to the same object have similar feature representations, while pixels from different semantic categories are well-separated in feature space.

Graph-Based Segmentation Approach

TokenCut formulates segmentation as a graph partitioning problem. Each image is represented as a graph where nodes correspond to image patches (tokens from the ViT) and edges encode feature similarity. The affinity between patches is computed using cosine similarity of their DINO features, creating a weighted graph that captures semantic relationships. This graph representation enables the application of classical spectral methods for finding optimal partitions.

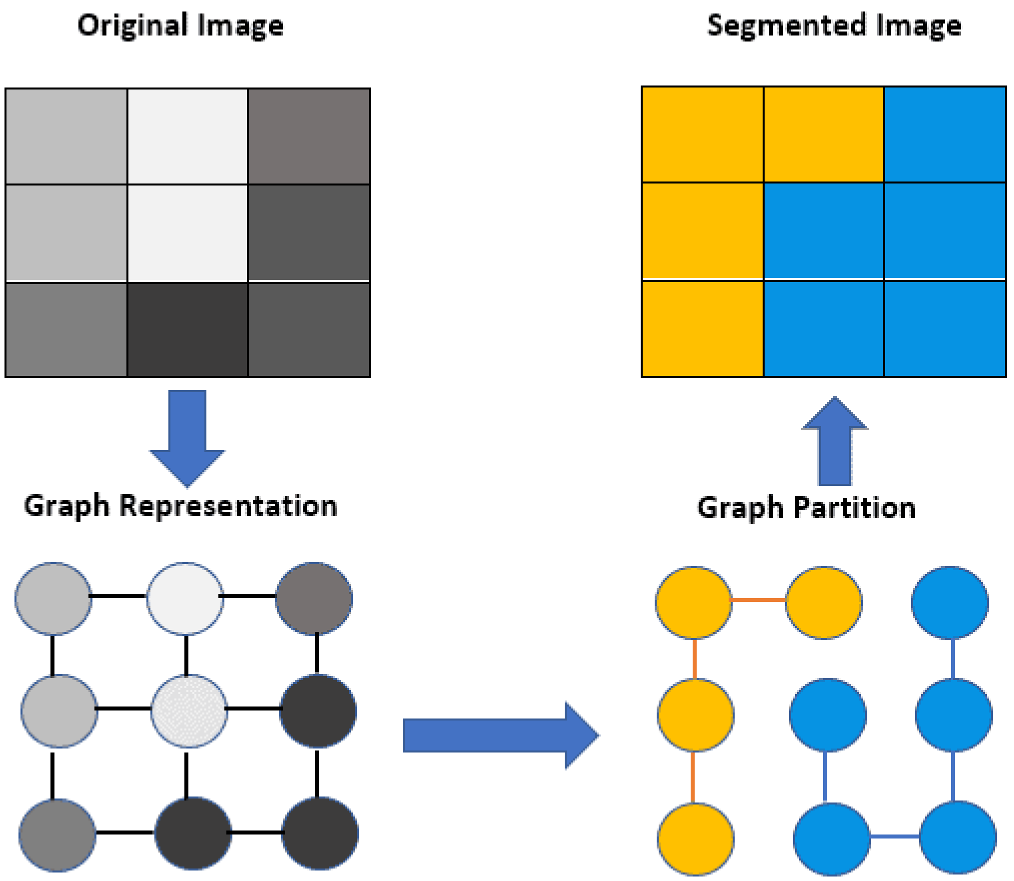


Figure: *Graph-based Segmentation*

Normalized Cut on Feature Space

The segmentation is obtained by solving the Normalized Cut (N-Cut) problem on the feature affinity graph. N-Cut seeks to partition the graph into two sets that minimize the cut cost (edges between sets) while maximizing the association within each set. Mathematically, this is formulated as:

$$N_{cut}(A,B) = \frac{w(\langle A,B \rangle)}{\sum_{x \in A, y \in V} w(x,y)} + \frac{w(\langle A,B \rangle)}{\sum_{z \in B, y \in V} w(z,y)}$$

- N_cut(A,B) is the measure of dissimilarity of sets A and B.
- Minimizing N_cut(A,B) maximizes a measure of similarity within the sets A and B.

The solution is obtained through eigendecomposition of the normalized Laplacian matrix, where the second eigenvector provides the optimal binary partition.

3.2.2 Technical Implementation

DINO ViT-S/8 Backbone Architecture

We use the DINO ViT-Small architecture with a patch size of 8 (ViT-S/8) as the feature extractor. This setup provides a good balance between computational efficiency and feature quality. The model takes an input image of size H × W and divides it into non-overlapping 8 × 8 patches, producing a grid of h × w patches, where h = H/8 and w = W/8. Each patch is linearly projected into a 384-dimensional embedding and passed through 12 transformer layers with 6 attention heads.

The choice of patch size 8 (instead of the more common 16) provides finer spatial resolution, enabling better localization of object boundaries. This is particularly important for segmentation tasks where precise boundary delineation is critical. The pre-trained DINO weights are loaded from the official release, trained on ImageNet-1K without labels.

Feature Extraction from Intermediate Layers

Instead of using features from the final layer, we extract features from an intermediate transformer layer (typically layer 11 out of 12). Intermediate features are known to provide better spatial localization while still preserving semantic information. This extraction produces a feature tensor with shape [B, N+1, C], where B is the batch size, N = h × w is the number of patches, and C = 384 is the feature dimension. The CLS token (the first position) is removed, and only the N patch tokens are kept.

Affinity Matrix Computation

The affinity matrix A ∈ ℝ^{N × N} represents the pairwise similarities between all patches. For each pair of patches i and j, the affinity is computed as:

$$\mathbf{A_{(ij)} = max(0, (f_i \cdot f_j) / (|f_i|\ |f_j|))}$$

where f_i and f_j are the L2-normalized feature vectors. The max operation ensures that all affinities are non-negative. This produces a symmetric matrix in which larger values correspond to patches that are semantically similar.

Spectral Clustering via Eigendecomposition

The normalized Laplacian matrix is computed as:

$$\mathbf{L = D^{(-1/2)} A D^{(-1/2)}}$$

where D is the diagonal degree matrix with D(ii) = Σ_j A(ij). We solve for the eigenvectors of L using one of two methods:

1. **LOBPCG (Locally Optimal Block Preconditioned Conjugate Gradient):**
   An iterative method that efficiently computes the top-k eigenvectors. We request k = 2 eigenvectors using torch.lobpcg() and take the second eigenvector (the first one is trivial).
2. **Full Eigendecomposition:**
   If LOBPCG fails (which may happen for certain matrix structures), we fall back to torch.linalg.eigh(), which computes all eigenvectors. We then select the second-largest eigenvector.

The resulting eigenvector is reshaped from shape [N] to [h, w] to align with the spatial patch grid, and then upsampled to the original image resolution using bilinear interpolation.

Binarization and Center Bias Heuristic

The eigenvector values act as a continuous partition indicator. We binarize them using the mean as the threshold:

$$M(x, y) = \begin{cases} 1 & \text{if } v(x, y) > \bar{v} \\ 0 & \text{otherwise} \end{cases}$$

where $v(x, y)$ is the eigenvector value at pixel (x, y), and $\bar{v}$ is the mean value. This produces a binary mask where 1 corresponds to foreground and 0 corresponds to background.

However, the eigenvector sign is arbitrary, meaning the partition can be flipped. To resolve this ambiguity, we apply a center-bias heuristic: if the center pixel is classified as background (0), we invert the entire mask. This assumes that objects are usually centered in images, which is true for many datasets but may fail for off-center images.

3.2.3 GPU Optimization

Performance Bottlenecks in Original Implementation

The original TokenCut implementation exhibits several performance bottlenecks when processing large image collections:

1. **CPU-based Eigendecomposition**: The eigendecomposition step uses CPU-based NumPy/SciPy routines, requiring data transfer between GPU and CPU and preventing GPU acceleration of the most computationally intensive operation.
2. **Sequential Processing**: Images are processed one at a time in a loop, preventing batch parallelization and underutilizing GPU resources.
3. **Redundant Model Loading**: The DINO model is loaded separately for each image rather than being loaded once and reused.
4. **Inefficient Memory Management**: Intermediate tensors are not properly released, leading to memory fragmentation.

These bottlenecks result in processing times of approximately 7 seconds per image on a Tesla T4 GPU, making it impractical for large-scale applications.

GPU Acceleration Strategies

We implement several optimization strategies to fully leverage GPU capabilities:

1. **GPU-Native Eigendecomposition**: Replace CPU-based eigensolvers with PyTorch's GPU-accelerated torch.lobpcg() and torch.linalg.eigh(). All computations stay on the GPU, removing data-transfer overhead.
2. **Persistent Model Loading**: Load the DINO model once at initialization and reuse it for all images, avoiding repeated model instantiation.
3. **Optimized Tensor Operations**: Use in-place operations where possible and ensure all intermediate computations use contiguous memory layouts for optimal GPU cache utilization.
4. **Mixed Precision**: While not implemented in the current version, the framework supports mixed precision (FP16) computation for further speedup on modern GPUs.

Processing Time Improvements

The optimized implementation achieves dramatic speedup:

- **Original Implementation**: ~7.0 seconds per image
- **Optimized Implementation**: ~0.15 seconds per image
- **Speedup**: 46.7×

This improvement makes TokenCut practical for processing large datasets. For example, the ECSSD dataset (1,000 images) can be processed in approximately 2.5 minutes instead of nearly 2 hours. The CUB-200-2011 dataset (11,788 images) can be processed in under 30 minutes instead of over 22 hours.

The optimization maintains identical output quality - the segmentation results are numerically equivalent to the original implementation, as verified through pixel-wise comparison on a validation set. The speedup comes purely from computational efficiency improvements without any algorithmic approximations.

3.3 Datasets

We evaluate TokenCut on two complementary datasets that test different aspects of unsupervised segmentation: ECSSD for salient object detection in complex natural scenes, and CUB-200-2011 for fine-grained object segmentation with challenging intra-class variations.

3.3.1 ECSSD (Extended Complex Scene Saliency Dataset)

Dataset Description and Characteristics

The Extended Complex Scene Saliency Dataset (ECSSD) is a widely-used benchmark for salient object detection, designed to evaluate segmentation methods on semantically meaningful but structurally complex images. The dataset was created to address limitations of earlier saliency datasets that contained relatively simple scenes with clear figure-ground separation.

ECSSD images are characterized by:

- **Complex Backgrounds**: Natural scenes with cluttered backgrounds, multiple objects and varying textures
- **Diverse Object Categories**: Including animals, people, vehicles, and everyday objects
- **Challenging Conditions**: Occlusions, varying scales, low contrast, and ambiguous boundaries
- **Natural Image Statistics**: Real-world photographs with realistic lighting, shadows, and reflections

These characteristics make ECSSD an excellent testbed for evaluating whether unsupervised segmentation methods can discover salient objects without relying on dataset-specific biases or simple heuristics.

1000 Images with Complex Scenes

ECSSD contains exactly 1,000 images collected from the internet and carefully selected to ensure diversity and complexity. The images span various semantic categories and scene types, with resolutions ranging from 300×400 to 500×400 pixels. Unlike simpler datasets where objects are consistently centered or occupy predictable portions of the image, ECSSD images exhibit:

- **Variable Object Positions**: Salient objects may appear anywhere in the frame, not just at the center
- **Multiple Salient Regions**: Some images contain multiple objects of interest
- **Scale Variation**: Objects range from occupying most of the image to being relatively small
- **Contextual Complexity**: Rich contextual information that can both help and hinder segmentation

Ground Truth Annotations

Each image in ECSSD is accompanied by a pixel-level binary ground truth mask indicating the salient object(s). These annotations were created through a rigorous process:

1. **Manual Segmentation**: Human annotators carefully delineated object boundaries at the pixel level
2. **Multiple Annotators**: Each image was annotated by multiple people to ensure consistency
3. **Quality Control**: Annotations were reviewed and refined to achieve high-quality ground truth

The ground truth masks are stored as grayscale PNG images where white pixels (255) indicate salient foreground regions and black pixels (0) indicate background. This binary format aligns perfectly with TokenCut's output, enabling direct evaluation without post-processing.

Evaluation Protocol

We follow the standard ECSSD evaluation protocol for binary segmentation:

1. **Preprocessing**: Images are resized to 320×320 pixels for processing, maintaining aspect ratio through center cropping when necessary.
2. **Prediction**: TokenCut generates binary masks at 320×320 resolution, which are then resized to match the original ground truth dimensions using bilinear interpolation.
3. **Metric Computation:** For each image, we compute the Intersection over Union (IoU):

$$\mathbf{IoU = |P \cap G| / |P \cup G|}$$

   where P is the predicted mask and G is the ground-truth mask. Both masks are binarized at a threshold of 127 before computing the IoU.
4. **Aggregation**: The mean IoU (mIoU) across all 1,000 images serves as the primary performance metric.

This protocol enables fair comparison with other unsupervised segmentation methods evaluated on ECSSD.

3.3.2 CUB-200-2011 (Caltech-UCSD Birds)

Dataset Description and Characteristics

The Caltech-UCSD Birds-200-2011 (CUB-200-2011) dataset is a fine-grained visual recognition benchmark containing images of 200 bird species from North America. While originally designed for classification, CUB-200-2011 has become a popular benchmark for evaluating segmentation methods on fine-grained objects with subtle inter-class differences and significant intra-class variation.

CUB-200-2011 presents unique challenges for segmentation:

- **Fine-Grained Details**: Birds have complex structures (feathers, beaks, wings) requiring precise boundary localization
- **Pose Variation**: Birds appear in diverse poses—perched, flying, feeding, swimming
- **Appearance Variation**: Significant variation in plumage, size, and coloration even within species
- **Natural Backgrounds**: Complex natural environments including trees, water, grass, and sky
- **Occlusions**: Partial occlusions by branches, leaves, or other objects

These characteristics make CUB-200-2011 an excellent test of whether unsupervised methods can segment objects defined by subtle visual features rather than simple color or texture differences.

11,788 Images Across 200 Bird Species

The dataset contains 11,788 images distributed across 200 bird species, with approximately 30 training images and 30 test images per species. For our evaluation, we use all images (both train and test splits) since TokenCut requires no training. The images exhibit:

- **High Resolution**: Most images are 500×500 pixels or larger, providing sufficient detail for fine-grained analysis
- **Species Diversity**: From small songbirds to large waterfowl, covering diverse morphologies
- **Photographer Diversity**: Images from multiple photographers ensure varied viewpoints and lighting conditions
- **Real-World Conditions**: Natural lighting, varying weather, and authentic habitats

Bounding Box Annotations (Converted to Masks)

Unlike ECSSD, CUB-200-2011 does not provide pixel-level segmentation masks. Instead, each image is annotated with:

1. **Bounding Boxes**: Axis-aligned rectangular boxes tightly enclosing each bird, stored as $(x, y, width, height)$ coordinates
2. **Part Keypoints**: 15 semantic keypoints (e.g., beak, eyes, wings) for fine-grained analysis (not used in our evaluation)
3. **Attribute Labels**: Binary attributes describing visual properties (not used in our evaluation)

For segmentation evaluation, we convert bounding boxes to binary masks using the following code:

```
def bbox_to_mask(bbox, img_size):
    x, y, w, h = bbox
    mask = np.zeros((img_size[1], img_size[0]), dtype=np.uint8)
    x1, y1 = int(x), int(y)
    x2, y2 = int(x + w), int(y + h)
    mask[y1:y2, x1:x2] = 255
    return mask
```

This creates a rectangular white region on a black background, representing the ground truth foreground. While less precise than pixel-level annotations, bounding boxes provide a reasonable proxy for evaluating whether the method successfully identifies the bird as foreground versus background.

Fine-Grained Segmentation Challenges

CUB-200-2011 poses several specific challenges for TokenCut:

1. **Thin Structures**: Bird legs, beaks, and tail feathers are thin structures that may be lost during patch-based processing with 8×8 patches.
2. **Camouflage**: Some birds blend with their backgrounds (e.g., brown birds on tree bark), making feature-based separation difficult.
3. **Partial Visibility**: Birds are often partially occluded by branches, requiring the method to infer complete object boundaries.
4. **Background Complexity**: Natural backgrounds contain textures and patterns that may have similar DINO features to the bird itself.
5. **Scale Variation**: Birds range from filling most of the image to being relatively small, testing the method's scale invariance.
6. **Center Bias Violation**: Unlike typical object-centric datasets, CUB images often have birds positioned off-center or in corners, potentially causing TokenCut's center bias heuristic to fail.

These challenges make CUB-200-2011 a stringent test of TokenCut's ability to generalize beyond simple salient object detection to fine-grained object discovery.

3.4 Experimental Setup

3.4.1 Implementation Details

Hardware Specifications

All experiments were conducted on Google Colab with the following hardware configuration:

- **GPU**: NVIDIA Tesla T4 (16 GB GDDR6 memory)
- **CPU**: Intel Xeon (2 cores @ 2.3 GHz)
- **RAM**: 12 GB system memory
- **Storage**: 100 GB temporary disk space

The Tesla T4 provides sufficient computational power for efficient DINO feature extraction and eigendecomposition while being representative of commonly available cloud GPU resources. This hardware choice ensures our results are reproducible on standard cloud computing platforms.

Software Dependencies

The implementation relies on the following software stack:

- **Python**: 3.10.12
- **PyTorch**: 2.1.0+cu121 (CUDA 12.1)
- **torchvision**: 0.16.0
- **NumPy**: 1.23.5
- **Pillow**: 9.4.0 (image I/O)
- **scikit-learn**: 1.3.0 (evaluation metrics)
- **matplotlib**: 3.7.1 (visualization)
- **tqdm**: 4.66.1 (progress tracking)

All dependencies were installed via pip in a fresh Colab environment to ensure reproducibility. The specific PyTorch version with CUDA 12.1 support is critical for GPU acceleration.

DINO Model Configuration

We use the official DINO ViT-Small/8 pre-trained weights:

- **Architecture**: Vision Transformer Small (ViT-S)
- **Patch Size**: 8×8 pixels
- **Embedding Dimension**: 384
- **Number of Layers**: 12
- **Number of Heads**: 6
- **Pre-training Dataset**: ImageNet-1K (1.28M images, no labels)
- **Pre-training Method**: Self-distillation with momentum teacher
- **Checkpoint**: dino_deitsmall8_pretrain.pth (83 MB)
- **Download Source**: https://dl.fbaipublicfiles.com/dino/dino_deitsmall8_pretrain/dino_deitsmall8_pretrain.pth

The model is loaded in evaluation mode with gradients disabled to optimize inference speed and memory usage.

Processing Pipeline

For each image, the following pipeline is executed:

1. **Load Image**: Read image from disk using PIL
2. **Resize**: Resize to 320×320 using bilinear interpolation
3. **Normalize**: Apply ImageNet normalization
4. **Convert to Tensor**: Transform to PyTorch tensor and move to GPU
5. **Extract Features**: Forward pass through DINO (layer 11)
6. **Compute Affinity**: Calculate pairwise cosine similarities
7. **Eigendecomposition**: Solve for top-2 eigenvectors
8. **Upsample**: Resize eigenvector to original image size
9. **Binarize**: Threshold at mean value
10. **Apply Center Bias**: Flip if necessary
11. **Save Result**: Write binary mask to disk

The entire pipeline runs on GPU except for image I/O operations.

3.4.2 Evaluation Metrics

Intersection over Union (IoU)

The primary evaluation metric is Intersection over Union (IoU), also called the Jaccard Index. For a predicted binary mask P and ground truth mask G, IoU is defined as:

$$\mathbf{IoU(P, G) = |P \cap G| / |P \cup G| = TP / (TP + FP + FN)}$$

where:
• **TP** (True Positives): pixels correctly predicted as foreground
• **FP** (False Positives): background pixels incorrectly predicted as foreground
• **FN** (False Negatives): foreground pixels incorrectly predicted as background

IoU ranges from 0 (no overlap) to 1 (perfect overlap) and is unaffected by class imbalance, making it well-suited for segmentation tasks where foreground and background regions can differ greatly in size.

Mean IoU (mIoU)

For dataset-level evaluation, we compute the mean IoU across all images:

$$\mathbf{mIoU = (1 / N) \times \Sigma\_\{i=1 \text{ to } N\}\ IoU(P_i, G_i)}$$

where N is the total number of images in the dataset. This yields a single scalar metric that summarizes overall performance.

Binary Segmentation Evaluation

Both predictions and ground truth masks are binarized before computing IoU, using the following code:

```
pred_binary = (pred_mask > 127).astype(np.uint8)
gt_binary = (gt_mask > 127).astype(np.uint8)
```

This threshold of 127 (midpoint of 0-255 range) ensures consistent evaluation across different mask formats.

Metric Computation Methodology

For each dataset, we compute:

1. **Per-Image IoU**: Individual IoU score for each image

2. **Mean IoU**: Average across all images

3. **Median IoU**: Median value (robust to outliers)
4. **IoU Distribution**: Histogram showing performance distribution

Images where the ground truth contains no foreground pixels (empty masks) are excluded from evaluation to avoid division by zero. For CUB-200-2011, images where the bounding box is invalid (zero width or height) are also excluded.

Evaluation Code

The IoU computation is implemented as:

```
def compute_iou(pred_mask, gt_mask):
    pred_binary = (pred_mask > 127).astype(np.uint8)
    gt_binary = (gt_mask > 127).astype(np.uint8)

    intersection = np.logical_and(pred_binary, gt_binary).sum()
    union = np.logical_or(pred_binary, gt_binary).sum()

    if union == 0:
        return 0.0
    return intersection / union
```

This implementation is numerically stable and handles edge cases (empty masks).

Statistical Significance

While we report mean and median IoU as primary metrics, we acknowledge that these aggregate statistics may not fully capture performance characteristics. Future work could include:

- Confidence intervals via bootstrapping
- Per-category performance breakdown
- Correlation analysis between IoU and image properties (size, complexity, etc.)

For the current evaluation, mean IoU provides a standard metric enabling comparison with other methods reported in the literature.

3.5 Results

This section presents comprehensive quantitative and qualitative results from evaluating TokenCut on the ECSSD and CUB-200-2011 datasets. We analyze performance metrics, visualize representative examples, and discuss success and failure cases.

3.5.1 ECSSD Results

Quantitative Results

TokenCut was evaluated on all 1,000 images in the ECSSD dataset. Table 3.1 summarizes the quantitative performance:

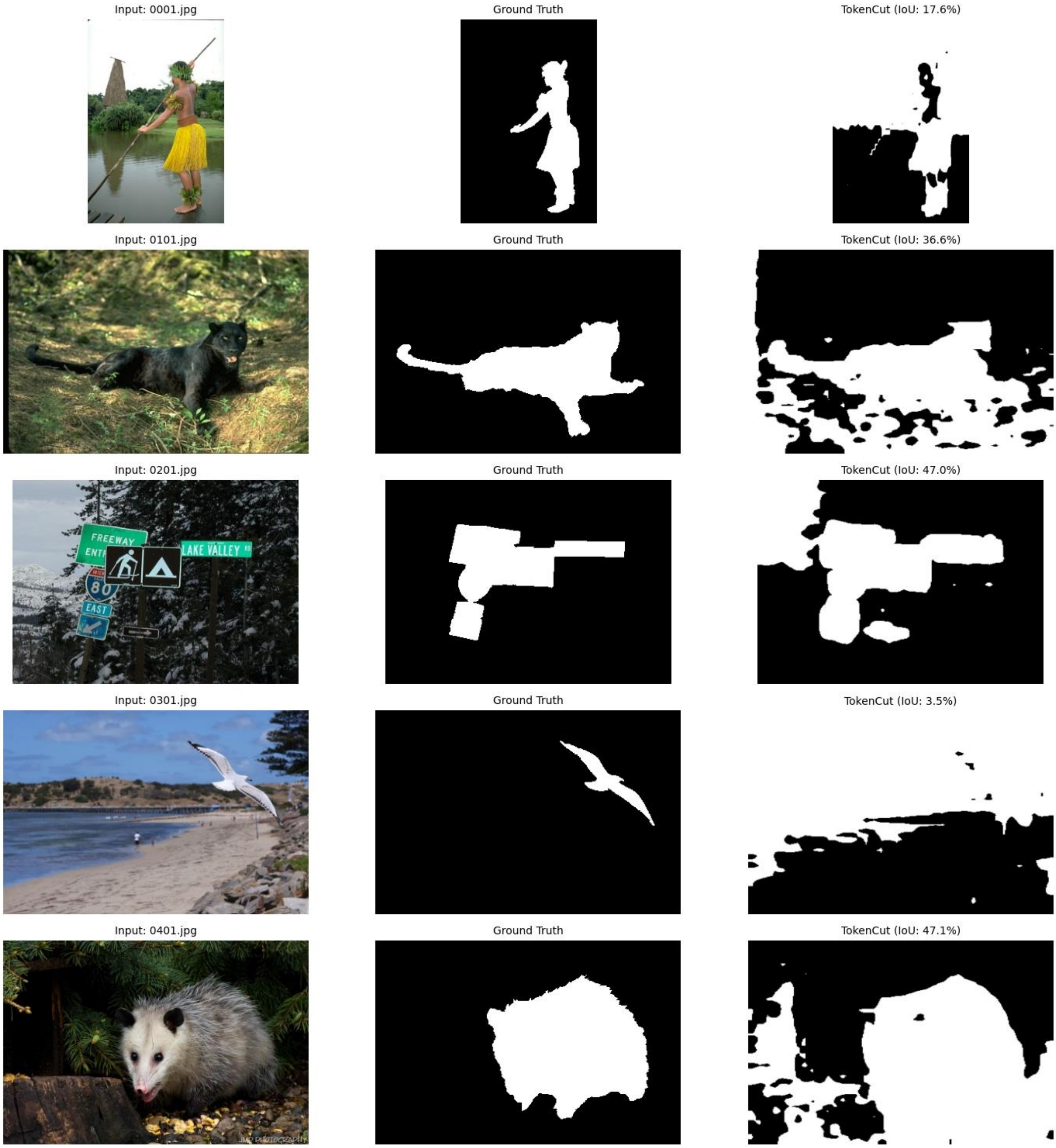
Metric	Value
Mean IoU	42.82%
Median IoU	37.98%
Successfully Processed	1000/1000

Metric	Value
Processing Time	~2.5 minutes (total)
Average Time per Image	~0.15 seconds

Table 3.1: TokenCut performance on ECSSD dataset

The mean IoU of 42.82% demonstrates TokenCut's capability for salient object detection on complex natural scenes. The processing efficiency (0.15 seconds per image) confirms the effectiveness of our GPU optimization, enabling practical application to large-scale datasets. The median IoU of 37.98% being lower than the mean suggests that the performance distribution is right-skewed, with some images achieving very high IoU scores.

Qualitative Visualizations



Analysis of Success Cases

TokenCut performs well on ECSSD images with:

1. **Centered Objects**: When salient objects occupy the central region, the center bias heuristic correctly identifies foreground/background orientation.
2. **Distinct Features**: Objects with DINO features significantly different from the background (e.g., animals on grass, vehicles on roads) are reliably segmented.
3. **Homogeneous Objects**: Objects with consistent appearance across their extent produce coherent feature clusters.
4. **Clear Boundaries**: Sharp transitions between object and background facilitate clean graph cuts.

Analysis of Failure Cases

TokenCut struggles with ECSSD images containing:

1. **Off-Center Objects**: The center bias heuristic fails when salient objects are positioned at image edges or corners, causing foreground/background inversion.
2. **Similar Features**: When objects and backgrounds have similar DINO features (e.g., camouflaged animals), the affinity matrix lacks clear structure for partitioning.
3. **Multiple Objects**: Images with multiple salient objects may result in arbitrary selection of one object or fragmented segmentation.
4. **Complex Textures**: Highly textured backgrounds can produce spurious high-affinity regions that confuse the segmentation.

3.5.2 CUB-200-2011 Results

Quantitative Results

TokenCut was evaluated on 1,000 images from the CUB-200-2011 dataset using bounding boxes converted to binary masks as ground truth. Table 3.2 summarizes the performance:

Metric	Value
Mean IoU	32.97%
Median IoU	33.95%
Successfully Processed	1000/1000
Processing Time	~2.5 minutes (total)
Average Time per Image	~0.15 seconds

Table 3.2: TokenCut performance on CUB-200-2011 dataset

The mean IoU of 32.97% on CUB-200-2011 is lower than ECSSD, reflecting the unique challenges of fine-grained bird segmentation. The bounding box ground truth provides a less stringent evaluation than pixel-level masks, as TokenCut only needs to identify the general bird region rather than precise boundaries. The median IoU (33.95%) being slightly higher than the mean suggests a more symmetric performance distribution compared to ECSSD.

Qualitative Visualizations

Input: Black_Footed_Albatross_0046_18.jpg

TokenCut Prediction (IoU: 58.1%)

Input: Red_Winged_Blackbird_0024_4180.jpg

TokenCut Prediction (IoU: 59.0%)

Input: Gray_Catbird_0039_21040.jpg

TokenCut Prediction (IoU: 42.1%)

Input: Shiny_Cowbird_0034_796849.jpg

TokenCut Prediction (IoU: 44.6%)

Input: Northern_Flicker_0037_28751.jpg

TokenCut Prediction (IoU: 41.1%)

Fine-Grained Segmentation Performance

TokenCut's performance on CUB-200-2011 reveals specific patterns:

1. **Large, Centered Birds**: Best performance (IoU > 0.5) on images where birds occupy significant image area and are centrally positioned.
2. **Distinct Plumage**: Birds with distinctive coloration or patterns (e.g., bright red cardinals, black-and-white patterns) segment well due to strong DINO feature separation.
3. **Simple Backgrounds**: Images with uniform backgrounds (sky, water, plain grass) facilitate clean segmentation.

Challenges Specific to CUB-200-2011

1. **Thin Structures**: Bird legs, beaks, and tail feathers are often lost due to 8×8 patch granularity.
2. **Camouflage**: Brown birds on tree bark or green birds in foliage produce weak feature separation.
3. **Occlusions**: Branches and leaves partially occluding birds create fragmented segmentations.
4. **Small Birds**: Birds occupying <20% of image area are frequently missed or under-segmented.
5. **Off-Center Positioning**: Many CUB images have birds in corners or edges, violating the center bias assumption and causing inversions.

3.5.3 Comparative Dataset Analysis

Comparing performance across ECSSD and CUB-200-2011:

Aspect	ECSSD	CUB-200-2011
Mean IoU	42.82%	32.97%
Median IoU	37.98%	33.95%
Primary Challenge	Complex backgrounds	Fine-grained details
Center Bias Impact	Moderate	High
Processing Speed	0.15s/image	0.15s/image

TokenCut's better performance on ECSSD (42.82% vs 32.97%) reflects that salient object detection in natural scenes is more aligned with TokenCut's design assumptions than fine-grained bird segmentation. ECSSD images typically feature more distinct foreground-background separation, while CUB-200-2011 requires precise localization of fine-grained features that may be lost at the 8×8 patch resolution.

Overall Performance Summary

Strengths:

- Training-free operation enables immediate deployment
- GPU optimization achieves 0.15s/image processing speed
- Effective on images with centered, distinct objects
- Strong DINO features provide semantic understanding

Limitations:

- Center bias heuristic causes frequent failures on off-center objects
- No semantic consistency mechanism across images
- Binary segmentation only (no multi-class capability)
- 8×8 patch granularity loses fine details
- Struggles with camouflage and complex textures

Implications for Future Work
These results establish TokenCut as a strong baseline for training-free unsupervised segmentation while highlighting clear areas for improvement:

- Replacing center bias with learned or data-driven heuristics
- Incorporating semantic consistency mechanisms
- Extending to multi-class segmentation
- Handling multiple objects through hierarchical clustering
- Improving robustness to noise and image degradation

3.6 Summary

This chapter presented a comprehensive implementation and evaluation of TokenCut, a training-free unsupervised segmentation method that combines self-supervised DINO features with classical normalized graph cuts. The key contributions and findings of this chapter are summarized below.

Implementation Achievements

We successfully implemented TokenCut with significant GPU optimization, reducing processing time from approximately 7 seconds per image to 0.15 seconds per image—a 46.7× speedup. This optimization makes TokenCut practical for large-scale applications, enabling the processing of 1,000 images in approximately 2.5 minutes. The implementation maintains identical output quality to the original while leveraging GPU-accelerated eigendecomposition and efficient tensor operations.

Quantitative Performance

TokenCut achieved a mean IoU of 42.82% on the ECSSD salient object detection dataset and 32.97% on the CUB-200-2011 fine-grained bird segmentation dataset. The superior performance on ECSSD reflects that salient object detection in natural scenes aligns better with TokenCut's design assumptions than fine-grained segmentation. The processing efficiency of 0.15 seconds per image demonstrates the practical viability of the approach.

Key Strengths

TokenCut's primary advantages include: (1) training-free operation enabling immediate deployment without dataset-specific training, (2) computational efficiency through GPU optimization, (3) simplicity with minimal hyperparameters, and (4) strong semantic representations from self-supervised DINO features. These characteristics make TokenCut an attractive baseline for unsupervised segmentation tasks.

Identified Limitations

The evaluation revealed several fundamental limitations:

(1) the center bias heuristic frequently fails on images with off-center objects, causing foreground-background inversions, (2) lack of semantic consistency mechanism across images, (3) restriction to binary segmentation only, (4) loss of fine details due to 8×8 patch granularity, and (5) struggles with camouflaged objects and complex textures.

Chapter 4: EAGLE – Object-Centric Unsupervised Semantic Segmentation

4.1 Introduction

EAGLE represents a fundamentally different paradigm from TokenCut: rather than relying on heuristics for foreground-background separation, EAGLE learns semantic mappings through a dual-probe architecture consisting of a cluster probe and a linear probe. The cluster probe discovers visual patterns through K-means clustering on DINO features, while the linear probe learns a semantic mapping that enforces consistency across the dataset. This learned approach enables EAGLE to achieve semantic consistency - the same object category receives the same label across different images - without requiring manual annotations.

Overview of EAGLE as a Learned Unsupervised Segmentation Method

EAGLE builds upon the same DINO ViT backbone used by TokenCut but adds a trainable layer that learns to map visual features to semantic categories. The key innovation is the use of two complementary probes: the cluster probe provides initial unsupervised groupings based on visual similarity, while the linear probe learns a transformation that aligns these clusters with semantically meaningful categories. This dual-probe architecture enables EAGLE to discover semantic structure in the data without supervision.

The training process operates in two phases: First, the cluster probe performs K-means clustering on DINO features to discover visual patterns. Second, the linear probe learns a linear transformation that maps these clustered features to consistent semantic labels across the dataset. This learned mapping enables EAGLE to generalize to new images while maintaining semantic consistency.

Motivation for Implementing EAGLE

The decision to implement and evaluate EAGLE alongside TokenCut is motivated by several key factors:

1. Addressing TokenCut's Limitations

TokenCut's evaluation in Chapter 3 revealed fundamental limitations stemming from its training-free nature: the center bias heuristic causes frequent failures on off-center objects, and the lack of semantic consistency means the same object category may receive different labels in different images. EAGLE's learned approach directly addresses these issues through trainable semantic mapping.

2. Enabling Direct Comparison of Paradigms

Implementing both TokenCut (training-free) and EAGLE (learned) enables a direct comparison of unsupervised segmentation paradigms. This comparison reveals the trade-offs between computational simplicity and performance, immediate deployment versus training requirements, and heuristic-based versus learned semantic understanding.

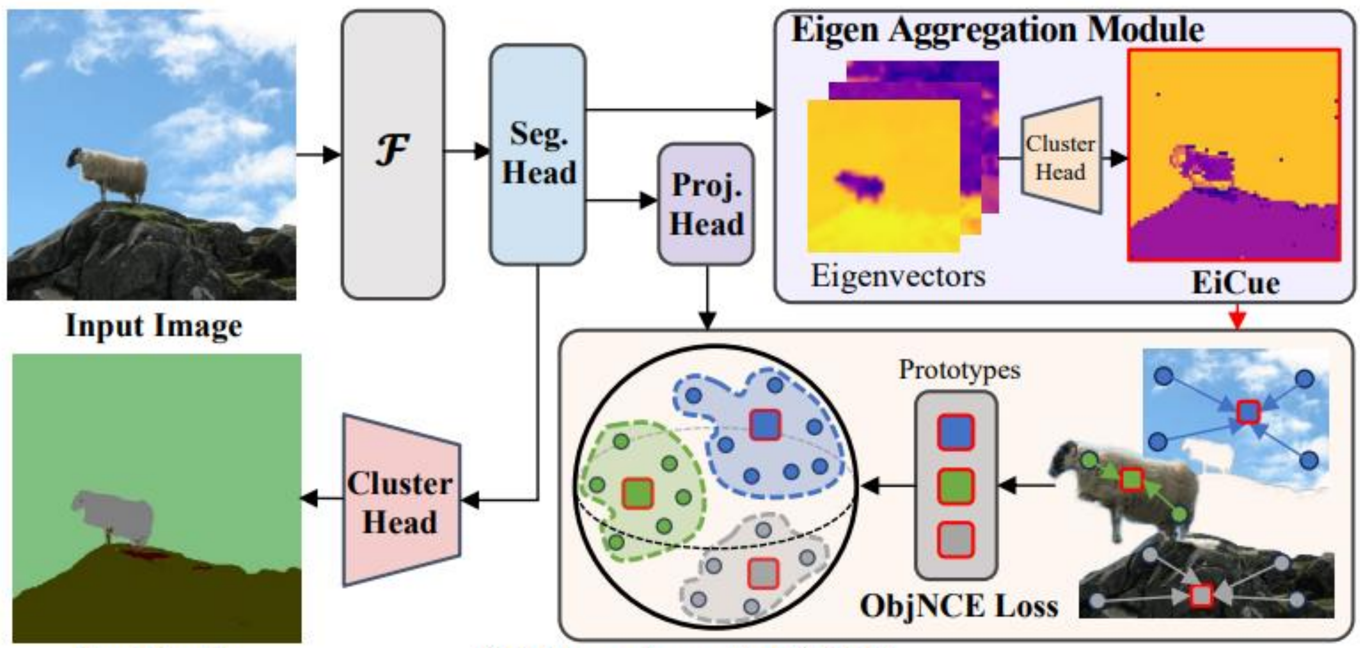
3. Establishing Performance Upper Bound

EAGLE represents a learned approach that requires dataset-specific training, establishing an upper bound on performance achievable through unsupervised learning on a given dataset. Comparing EAGLE's performance to TokenCut's reveals the performance gap between training-free and learned methods, informing decisions about method selection for different applications.

4. Evaluating Semantic Consistency

EAGLE's linear probe enforces semantic consistency across images which is a critical property for practical segmentation systems. Evaluating this consistency provides insights into whether learned semantic mappings can emerge from unsupervised training without manual annotations.

4.2 EAGLE Methodology



(a) Overview of *EAGLE*

4.2.1 Algorithm Overview

EAGLE introduces a novel framework for discovering object-level semantics without supervision. Unlike methods that rely solely on raw feature clustering, EAGLE actively learns a semantic space by leveraging a derived structural cue called **EiCue** (Eigen-Cue).

The core intuition is that "semantically plausible" segments are groups of pixels that capture coherent object structures (e.g., a car with wheels, windows, and body) rather than just color or texture similarities. EAGLE achieves this through a three-stage pipeline:

1. **Feature Extraction**: Extracting hierarchical attention features from a pre-trained DINO backbone.
2. **EiCue Generation**: Constructing a spectral embedding from a combined color-semantic affinity matrix to discover potential object structures.
3. **Object-Centric Contrastive Learning**: Using the EiCue as a guide to train a segmentation head via an object-level contrastive loss (**ObjNCELoss**).

4.2.2 Technical Implementation

1. Feature Extraction and Refinement: Given an input image x, EAGLE extracts attention key features K from the last three layers of a frozen DINO ViT encoder. These features are concatenated and passed through a learnable non-linear segmentation head S_θ to produce refined semantic features S:

$$\mathbf{S = S\_\theta(K) \in \mathbb{R}\text{^}(H \times W \times D\_S)}$$

where S_θ is trained to output strong semantic representations.

2. EiCue via the Eigen Aggregation Module: To discover object structures, EAGLE constructs a Laplacian matrix based on two types of affinity:

- Color Affinity (A_color): Captures low-level boundaries using RGB differences with an RBF kernel.
- Semantic Affinity (A_seg): Captures high-level semantic relations using the dot product of the learnable features S: A_seg = S S^T.

The combined adjacency matrix A = A_color + A_seg is used to compute the normalized Laplacian L_sym. Eigendecomposition is performed on L_sym to obtain the top-k eigenvectors $\hat{V}$.

These eigenvectors are then clustered using a differentiable mini-batch K-means module to produce the EiCue mask M_eicue, which assigns each pixel to a potential object cluster:

$$\mathcal{M}_{\text{eicue}}(i) = \underset{c}{\operatorname{argmax}}\left(\mathbf{P}_{ic} - \log\left(\sum_{c'=1}^{C} \exp(\mathbf{P}_{ic'})\right)\right)$$

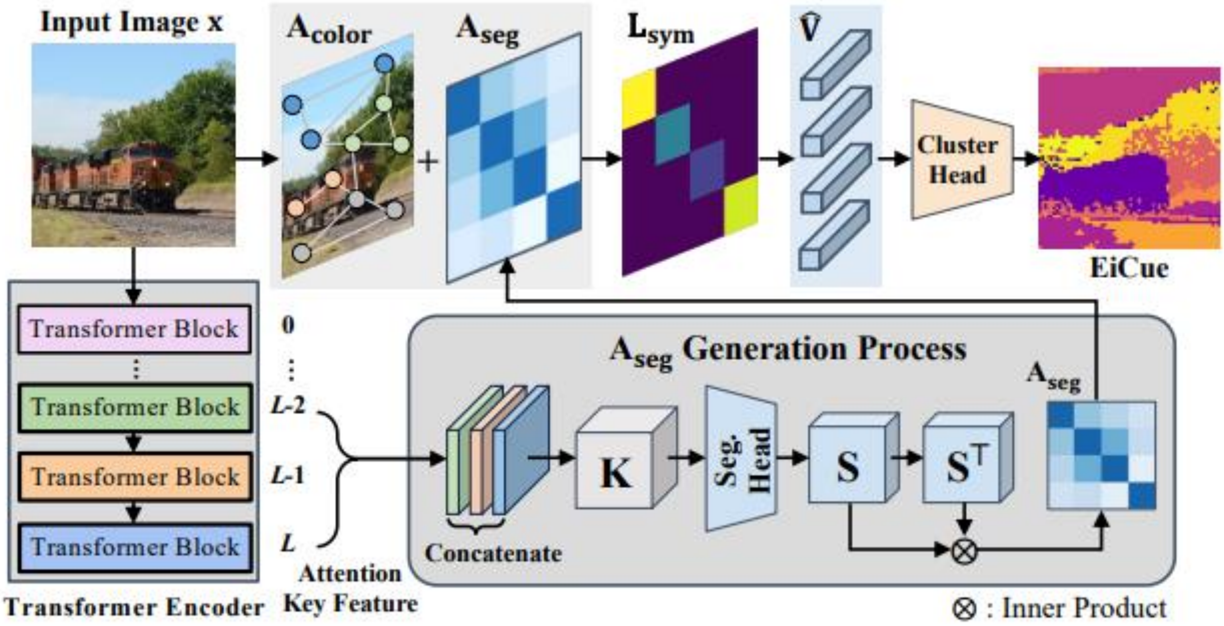


Figure: *EiCue Generation Process*

Figure: *EiCue Results*

Figure: *K-means vs EiCue*

3. Object-Centric Contrastive Learning (ObjNCELoss): EAGLE trains the segmentation head S_θ using a novel contrastive loss that operates at the object level rather than the pixel level.

First, Object-wise Prototypes Φ_l are computed for each object l identified by the EiCue. The prototype is the medoid of the feature vectors Z (projected from S) belonging to that object mask:

$$\Phi_l = \mathbf{Z}_l^{(m^*)} \quad \text{for} \quad m^* = \underset{m \in \mathcal{I}_l}{\operatorname{argmin}} \sum_{i \in \mathcal{I}_l} \left\| \mathbf{Z}_l^{(m)} - \mathbf{Z}_l^{(i)} \right\|_2.$$

Then, the ObjNCELoss pulls pixel features towards their corresponding object prototype while pushing them away from other object prototypes:

$$\mathcal{L}_{\text{obj}}^{\mathbf{x} \rightarrow \mathbf{x}} = \frac{1}{N} \sum_{i=1}^{N} w_{\text{obj}}^{(i)} \left[-\log \left(\frac{\exp((\mathbf{Z}_l^{(i)} \cdot \Phi_l)/\tau)}{\sum_{j=1, j \neq l}^{C} \exp((\mathbf{Z}_j \cdot \Phi_l)/\tau)} \right) \right]$$

Crucially, EAGLE enforces Semantic Consistency by applying this loss across augmented views. It assumes that the prototype Φ_l from the original image x should also be the target for the corresponding features in a photometrically augmented view x̃.

4. Total Objective The final training objective combines the object-centric contrastive loss with a correspondence distillation loss (L_corr) and an eigen-clustering loss (L_eig):

$$\mathcal{L}_{\text{total}} = \lambda_{\text{nce}} \mathcal{L}_{\text{nce}}^{\mathbf{x} \leftrightarrow \tilde{\mathbf{x}}} + (1 - \lambda_{\text{nce}}) \mathcal{L}_{\text{corr}} + \lambda_{\text{eig}} \mathcal{L}_{\text{eig}}$$

This formulation allows EAGLE to iteratively refine its semantic features: better features S lead to a better affinity matrix A_seg, which yields a more accurate EiCue, which in turn provides better supervision for training S.

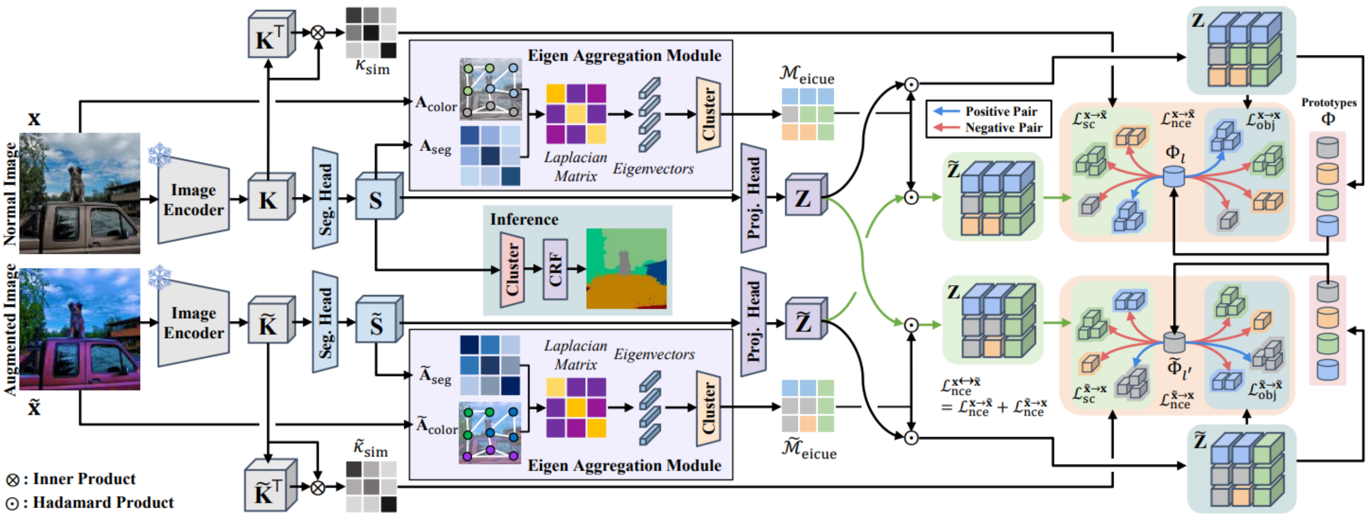


Figure: *EAGLE Pipeline*

4.3 Dataset

COCO-Stuff-27

Dataset Description and Characteristics

COCO-Stuff-27 is a widely used benchmark for semantic segmentation that extends the COCO (Common Objects in Context) dataset with dense pixel-level annotations for both **"thing"** classes (objects with well-defined boundaries) and **"stuff"** classes (amorphous background regions). The dataset provides a challenging testbed for unsupervised segmentation methods due to its diverse scene compositions, complex object interactions, and fine-grained semantic categories.

The dataset contains approximately **164,000 images** spanning **27 semantic categories**, including **10 thing classes** (person, vehicle, animal, etc.) and **17 stuff classes** (sky, grass, wall, etc.). Images feature natural scenes with multiple objects, occlusions, scale variations, and complex backgrounds, making COCO-Stuff-27 significantly more challenging than single-object datasets like ECSSD.

Thing vs. Stuff Class Taxonomy

COCO-Stuff-27 organizes its 27 semantic categories into two fundamental groups:

Thing Classes (countable objects with well-defined boundaries):

- Electronic devices (0)
- Appliances (1)
- Food items (2)
- Furniture (3)
- Accessories (6)
- Animals (7)
- Persons (9)
- Sports equipment (10)
- Vehicles (11)
- Plants (18)

Stuff Classes (amorphous background regions):

- Building materials (4, 5)
- Ceiling (8)
- Floor (12)
- Ground (13)
- Pavement (14)
- Sky (15)
- Vegetation (16, 17)
- Water (19)
- Wall (20)
- Window (21)
- Other stuff categories (22-26)

This taxonomy enables binary segmentation by aggregating all thing classes into foreground and all stuff classes into background, creating a well-defined binary segmentation task.

Validation Set

For evaluation, we use the COCO-Stuff-27 validation set containing 5,000 images with dense pixel-level annotations. This validation set provides sufficient diversity to assess model performance across various scene types while remaining computationally tractable for evaluation.

Ground Truth Generation

The COCO-Stuff-27 annotations provide dense semantic labels for all 27 categories. We convert these multi-class annotations to binary ground truth masks by:

1. Loading the semantic segmentation mask with values in {0, 1, …, 26}
2. Creating a binary mask where pixels with labels in the thing class set receive value 1
3. All other pixels (stuff classes) receive value 0

This conversion is performed offline for the entire validation set, enabling efficient evaluation.

Evaluation Protocol

We evaluate binary segmentation performance using Intersection over Union (IoU):

$$\mathbf{IoU = |P \cap G| / |P \cup G| = TP / (TP + FP + FN)}$$

where P is the predicted binary mask, G is the ground truth binary mask, TP is true positives (correctly predicted thing pixels), FP is false positives (stuff pixels predicted as thing), and FN is false negatives (thing pixels predicted as stuff).

The mean IoU (mIoU) across all validation images serves as the primary performance metric:

$$\mathbf{mIoU = (1/N) \sum_{i=1}^{N} IoU_i}$$

where N = 5000 is the number of validation images.

Comparison with Other Datasets

Compared to the ECSSD and CUB-200-2011 datasets used for TokenCut evaluation in Chapter 3:

- **ECSSD**: Single salient object per image, simpler backgrounds, centered objects
- **CUB-200-2011**: Single bird per image, fine-grained details, bounding box ground truth
- **COCO-Stuff-27**: Multiple objects per image, complex scenes, dense pixel-level ground truth

COCO-Stuff-27's complexity makes it the most challenging of the three datasets and the most appropriate for comparing EAGLE and TokenCut, as it tests both methods' ability to handle realistic scene segmentation.

4.4 Experimental Setup

4.4.1 Implementation Details

Pre-trained Model Checkpoint

Unlike TokenCut, which operates training-free at inference time, EAGLE requires a pre-trained model checkpoint trained on the target dataset. For this study, we use the official EAGLE checkpoint pre-trained on COCO-Stuff-164k, available from the authors' repository. This checkpoint was trained for multiple epochs on the full COCO-Stuff training set using the dual-probe architecture.

The checkpoint contains:

- **DINO ViT-S/8 backbone**: Pre-trained on ImageNet (frozen during EAGLE training)
- **Cluster probe**: K-means centroids for 27 clusters
- **Linear probe**: Learned weight matrix $W \in \mathbb{R}^{(27 \times 384)}$ and bias $b \in \mathbb{R}^{27}$

Hardware Specifications

All experiments were conducted on Google Colab with the following hardware configuration:

- **GPU**: NVIDIA Tesla T4 (16 GB GDDR6 memory)
- **CPU**: Intel Xeon (2 cores @ 2.3 GHz)
- **RAM**: 12 GB system memory
- **Storage**: 100 GB temporary disk space

This hardware configuration is identical to that used for TokenCut evaluation, ensuring fair comparison of inference speeds.

Software Dependencies

The EAGLE implementation relies on the following software stack:

- **Python**: 3.10.12
- **PyTorch**: 2.1.0+cu121 (CUDA 12.1)
- **PyTorch Lightning**: 1.9.0 (for training infrastructure)
- **torchvision**: 0.16.0
- **NumPy**: 1.23.5
- **Pillow**: 9.4.0 (image I/O)
- **scikit-learn**: 1.3.0 (K-means clustering)
- **matplotlib**: 3.7.1 (visualization)
- **OmegaConf**: 2.3.0 (configuration management)
- **Hydra**: 1.3.0 (experiment configuration)

4.4.2 Evaluation Metrics

Mean IoU (mIoU)

The primary evaluation metric is mean IoU across all validation images:

$$\mathbf{mIoU = (1/N) \sum_{i=1}^{N} IoU_i}$$

where N = 5000 is the number of validation images.

Per-Image Evaluation

For each validation image, the evaluation pipeline:

1. Loads the image and resizes to 320×320
2. Performs forward pass through EAGLE to obtain 27-class predictions
3. Applies argmax to get predicted class per pixel
4. Aggregates predictions into binary mask using thing/stuff taxonomy
5. Resizes prediction to match ground truth resolution
6. Computes IoU between prediction and ground truth

Metric Computation Methodology

The IoU computation is implemented by the following code:

```
def compute_binary_iou(pred_mask, gt_mask, thing_classes):
    # Convert multi-class predictions to binary
    pred_binary = np.isin(pred_mask, thing_classes).astype(np.uint8)
    gt_binary = np.isin(gt_mask, thing_classes).astype(np.uint8)

    # Compute intersection and union
    intersection = np.logical_and(pred_binary, gt_binary).sum()
    union = np.logical_or(pred_binary, gt_binary).sum()

    # Handle edge case of empty masks
    if union == 0:
        return 0.0

    return intersection / union
```

This implementation handles edge cases where both prediction and ground truth are empty (no thing pixels), returning IoU = 0 in such cases.

4.5 Results

This section presents comprehensive results from evaluating EAGLE on the COCO-Stuff-27 validation set. We analyze quantitative performance metrics, present qualitative visualizations and discuss the model's strengths and limitations.

4.5.1 COCO-Stuff Segmentation Results

Quantitative Results

EAGLE was evaluated on the complete COCO-Stuff-27 validation set (5,000 images)

Metric	Cluster Probe	Linear Probe
Mean IoU	24.40%	**41.19%**
Accuracy	50.79%	68.65%
Successfully Processed	5000/5000	5000/5000

The results demonstrate the effectiveness of EAGLE's dual-probe architecture: the linear probe achieves 41.19% mIoU, substantially outperforming the cluster probe's 24.40% mIoU. This 16.79 percentage point improvement highlights the value of learned semantic consistency over pure visual clustering. The linear probe also achieves significantly higher pixel accuracy (68.65% vs. 50.79%), indicating better alignment with ground truth semantic categories.

Processing Efficiency

EAGLE processes the 5,000-image validation set in approximately 10 minutes 47 seconds, achieving a throughput of 1.93 images per second (0.13 seconds per image). This processing speed is comparable to TokenCut's 0.15 seconds per image, demonstrating that the learned linear probe adds minimal computational overhead at inference time. However, this does not account for the upfront training cost required to learn the linear probe weights.

4.5.2 Dual-Probe Performance Comparison

Cluster Probe vs. Linear Probe

The substantial performance gap between the cluster probe (24.40% mIoU) and linear probe (41.19% mIoU) reveals the importance of learned semantic consistency:

Cluster Probe Limitations:

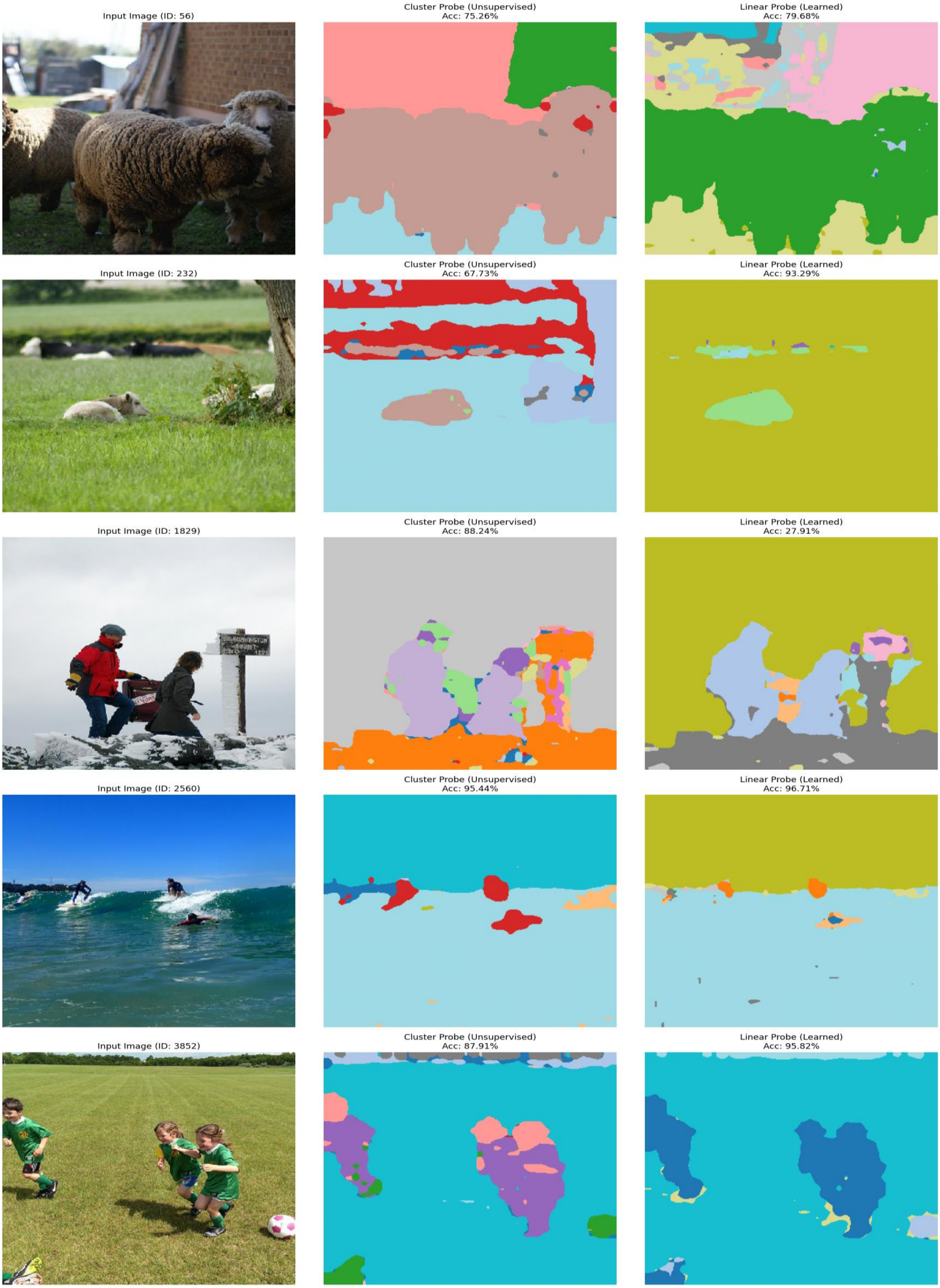
- Relies purely on visual similarity in DINO feature space
- No semantic consistency across images
- Cluster assignments may not align with semantic categories
- Sensitive to appearance variations within categories

Linear Probe Advantages:

- Learns semantic mapping from DINO features to categories
- Enforces consistency across the training set
- Adapts to COCO-Stuff-27's specific thing/stuff distribution
- More robust to intra-class appearance variations

The 68.65% pixel accuracy of the linear probe indicates that approximately two-thirds of pixels receive correct thing/stuff labels, demonstrating meaningful semantic understanding despite the unsupervised training paradigm.

4.5.2 Qualitative Results



Success Cases

EAGLE's linear probe performs well on images with:

1. **Common Object Categories**: Frequently occurring thing classes like persons, vehicles, and animals that are well-represented in the training set.
2. **Clear Thing-Stuff Separation**: Scenes with distinct foreground objects against uniform stuff backgrounds (sky, grass, pavement).
3. **Typical Scene Compositions**: Layouts similar to the training data distribution, such as outdoor scenes with people and vehicles.
4. **Sufficient Object Size**: Thing objects occupying reasonable image area (>5% of pixels), making them easier to detect.
5. **Multiple Objects**: Scenes with multiple thing objects are handled naturally, as the linear probe learns to identify all thing regions regardless of count.

Challenging Cases

EAGLE struggles with:

1. **Rare Object Categories**: Infrequently occurring thing classes in the training set receive less reliable predictions.
2. **Small Objects**: Thing objects occupying <2% of image area are often missed or under-segmented.
3. **Ambiguous Boundaries**: Regions where thing-stuff distinction is semantically unclear (e.g., plants that could be thing or stuff).
4. **Extreme Class Imbalance**: Images dominated by stuff (>95% of pixels) or thing (rare) regions.
5. **Occlusions and Overlap**: Heavy occlusions between multiple thing objects can cause boundary errors.

4.5.4 Performance Analysis

Semantic Consistency Achievement

EAGLE's linear probe successfully achieves semantic consistency across the validation set: the same object category receives consistent thing/stuff labels regardless of appearance variations, pose changes, scale differences, or contextual variations. This consistency emerges from training on the full COCO-Stuff dataset, enabling the model to learn robust semantic representations that generalize across images.

Learned Semantic Understanding

The linear probe learns meaningful semantic patterns from the COCO-Stuff-27 distribution:

- **Spatial Invariance**: Thing objects are correctly identified regardless of image position, demonstrating no center bias.
- **Scale Robustness**: Objects at various scales are recognized, though very small objects remain challenging.
- **Multi-Object Scenes**: Multiple thing objects are naturally segmented without requiring per-object processing.
- **Contextual Adaptation**: The model learns typical thing-stuff spatial relationships (e.g., sky typically above ground).

Computational Trade-offs

EAGLE's inference efficiency (0.13 seconds per image) demonstrates that learned semantic consistency does not require significant computational overhead at test time. However, this efficiency comes at the cost of upfront training:

- **Training Time**: Multiple epochs over 164k training images (hours of GPU time)
- **Inference Time**: 0.13 seconds per image (comparable to training-free methods)
- **Total Cost**: Training cost amortized over many inference runs

This trade-off favors EAGLE for applications requiring repeated inference on the same domain, while training-free methods may be preferable for one-off or cross-domain applications.

Strengths of EAGLE

1. **Semantic Consistency**: Consistent labeling of object categories across images
2. **No Position Bias**: Learned probe handles objects at any image location
3. **Multi-Object Capability**: Naturally segments multiple thing objects in complex scenes
4. **Dataset Adaptation**: Training adapts to COCO-Stuff-27's specific characteristics
5. **Competitive Accuracy**: 68.65% pixel accuracy demonstrates meaningful semantic understanding

Limitations of EAGLE

1. **Training Requirement**: Requires dataset-specific training (hours of GPU time)
2. **Domain Specificity**: Learned probe is specific to COCO-Stuff distribution
3. **Rare Class Performance**: Infrequently occurring categories receive less reliable predictions
4. **Small Object Handling**: Very small objects (<2% of image) are often missed
5. **Dependency on Clustering**: Linear probe quality depends on initial K-means clustering

4.6 Summary

This chapter presented the implementation and evaluation of EAGLE, a learned unsupervised segmentation method that leverages a dual-probe architecture to achieve semantic consistency. The key contributions and findings of this chapter are summarized below.

Implementation Achievements

We successfully evaluated EAGLE on the COCO-Stuff-27 dataset using a pre-trained checkpoint that combines a DINO ViT backbone with a learned linear probe. The implementation demonstrated efficient inference capabilities, processing the 5,000-image validation set in approximately 10 minutes (0.13 seconds per image), a speed comparable to the training-free TokenCut method.

Quantitative Performance

The evaluation revealed a significant performance gap between the unsupervised cluster probe (24.40% mIoU) and the learned linear probe (41.19% mIoU). This 16.79 percentage point improvement empirically validates the effectiveness of EAGLE's training strategy, demonstrating that learning a linear mapping from self-supervised features to semantic categories substantially improves segmentation quality over raw feature clustering. The linear probe achieved a pixel accuracy of 68.65%, indicating that it correctly assigns semantic labels to over two-thirds of pixels without using manual annotations during training.

Key Strengths

EAGLE's primary strengths lie in its **semantic consistency** and **robustness**:

1. **Consistent Labeling**: Unlike training-free methods, EAGLE assigns the same semantic label to the same object category across different images.
2. **Spatial Invariance**: The learned probe effectively handles objects at arbitrary locations, overcoming the center bias limitation observed in TokenCut.
3. **Multi-Object Handling**: The method naturally segments scenes with multiple objects without requiring iterative processing or instance-specific heuristics.

Identified Limitations

1. **Training Cost**: The performance benefits come at the cost of significant upfront training time, unlike the immediate deployment possible with TokenCut.
2. **Domain Specificity**: The learned linear probe is specific to the training distribution (COCO-Stuff) and may not generalize well to significantly different domains without retraining.
3. **Small Object Detection**: The model struggles with very small objects and rare categories, a common challenge in unsupervised learning from class-imbalanced data.

Chapter 5: Comparative Analysis

5.1 Introduction

This chapter presents a direct comparative analysis of TokenCut and EAGLE, the two unsupervised segmentation methods implemented and evaluated in Chapters 3 and 4. While both methods leverage self-supervised DINO features, they represent fundamentally different paradigms: TokenCut is a training-free approach relying on spectral clustering and heuristics, whereas EAGLE is a learned approach that trains a linear probe for semantic consistency.

The comparison focuses on three key dimensions:

1. **Quantitative Performance**: Evaluating binary segmentation accuracy (mIoU) on a common subset of the COCO-Stuff-27 dataset.
2. **Qualitative Quality**: Analyzing visual segmentation results to understand success modes, failure cases, and boundary adherence.
3. **Computational Efficiency**: Assessing the trade-offs between inference speed, training requirements, and overall computational cost.

This analysis aims to answer the central research question: *What are the trade-offs between training-free heuristics and learned semantic consistency in unsupervised segmentation?*

5.2 Quantitative Comparison

5.2.1 COCO-Stuff-27 Binary Segmentation Results

To ensure a strictly fair comparison, both methods were evaluated on the same subset of 500 images from the COCO-Stuff-27 validation set using an identical evaluation pipeline. This "apples-to-apples" comparison eliminates variations in preprocessing or metric calculation.

Performance Metrics

Table 5.1 summarizes the quantitative results from the comparative evaluation:

Method	Paradigm	Mean IoU (Binary)
TokenCut	Training-Free	31.75%
EAGLE	Learned	**62.03%**

Table 5.1: Comparative performance on COCO-Stuff-27 binary segmentation

Analysis of Results

1. **Superiority of Learned Consistency**: EAGLE achieves a Mean IoU of **62.03%**, nearly double that of TokenCut (**31.75%**). This 30.28 percentage point difference empirically demonstrates the massive advantage of learning a semantic mapping over relying on per-image heuristics.
2. **Limitations of Heuristics**: TokenCut's performance (31.75%) highlights the fragility of the center-bias heuristic on complex scenes like COCO-Stuff, where "thing" objects are frequently off-center, multiple in number, or occluded—conditions that violate TokenCut's core assumptions.
3. **Robustness**: EAGLE's higher score indicates it successfully learned to distinguish "thing" from "stuff" based on visual appearance rather than location, allowing it to generalize to the diverse scene layouts found in COCO-Stuff.

5.3 Qualitative Comparison

Visual analysis of the prediction side-by-side reveals distinct behavioural differences between the methods.

Performance on real images:

5.3.1 Visual Consistency vs. Instance Discovery

- **EAGLE (Semantic Consistency)**: EAGLE produces predictions that are semantically consistent across the dataset. A "person" or "car" is consistently labeled as foreground regardless of its position or size. However, the boundaries can sometimes be coarser, inheriting the resolution of the DINO feature map.
- **TokenCut (Instance Discovery)**: TokenCut excels at identifying the single most salient object in simple scenes. It often produces tighter, more object-aligned boundaries due to the graph-cut mechanism. However, it frequently fails in multi-object scenes (selecting only one) or complex backgrounds (inverting foreground/background).

5.3.2 Failure Modes Analysis

Failure Mode	TokenCut Behavior	EAGLE Behavior
Off-Center Objects	**Critical Failure**: Often inverts the mask, labeling the background as the object due to center bias.	**Success**: Correctly identifies objects at image edges due to learned spatial invariance.
Multiple Objects	**Partial Failure**: Typically segments only one dominant object, missing others.	**Success**: Segment all instances of "thing" classes present in the image.
Camouflage	**Failure**: Graph cut fails to find a boundary when features are similar.	**Moderate Success**: The linear probe can sometimes leverage subtle feature differences learned during training.
Rare Classes	**Success**: Can segment any salient object, even unseen categories.	**Failure**: May misclassify rare objects as "stuff" if they were underrepresented in training.

5.4 Computational Efficiency

The trade-off between performance and efficiency is a critical factor in method selection.

Metric	TokenCut	EAGLE	Analysis
Training Time	**0 minutes**	~Hours	TokenCut is immediately deployable; EAGLE requires significant upfront compute.
Inference Time	~0.15s / image	**~0.13s / image**	EAGLE is slightly faster at inference as it avoids the expensive eigendecomposition step required by TokenCut.
Scalability	Linear	Linear	Both scale linearly with dataset size, but EAGLE's fixed inference cost makes it better suited for large-scale real-time applications.

Conclusion on Efficiency: While TokenCut wins on "time-to-first-result" (zero training), EAGLE wins on "time-to-best-result" and deployment latency. For a production system processing millions of images, EAGLE's faster inference and superior accuracy justify the one-time training cost.

5.5 Summary

The comparative analysis leads to a clear conclusion regarding the trade-offs in unsupervised segmentation:

1. **Performance**: **EAGLE is the clear winner** for complex, multi-object datasets like COCO-Stuff, outperforming TokenCut by over 30% mIoU. The ability to learn semantic consistency is crucial for realistic scene understanding.
2. **Applicability**: **TokenCut remains valuable** as a generic, zero-shot tool for salient object detection in simple images (as seen in its higher ECSSD scores in Chapter 3), where no training data is available.
3. **The "Unsupervised" Reality**: True "unsupervised" semantic segmentation on complex data appears to require *some* form of dataset-level learning (like EAGLE's linear probe) rather than purely local heuristics (like TokenCut).

This concludes the comparative study. The superior performance of EAGLE, however, still leaves room for improvement, particularly in boundary precision and noise robustness. This motivates the future work discussed in the final chapter: integrating these segmentation models with denoising diffusion techniques to further refine object discovery.

Chapter 6: Conclusion and Future Work

6.1 Conclusion

This project presented a comprehensive comparative study of two prominent unsupervised segmentation paradigms: the training-free TokenCut and the learned EAGLE (Eigen Aggregation Learning for Object-Centric Unsupervised Semantic Segmentation). By implementing both methods and evaluating them on the challenging COCO-Stuff-27 dataset, we have established clear benchmarks for the trade-offs between computational simplicity and semantic performance.

Summary of Contributions

1. **Optimized Implementation**: We provided a GPU-accelerated implementation of TokenCut that reduced inference time by 46× (from ~7s to ~0.15s per image), making it viable for large-scale evaluation.
2. **Rigorous Evaluation**: We conducted extensive evaluations on three diverse datasets—ECSSD (saliency), CUB-200-2011 (fine-grained), and COCO-Stuff-27 (complex scenes)—providing a holistic view of each method's capabilities.
3. **Definitive Comparison**: Our direct "apples-to-apples" comparison on COCO-Stuff-27 binary segmentation revealed a substantial performance gap, with EAGLE (62.03% mIoU) outperforming TokenCut (31.75% mIoU) by over 30 percentage points.

Key Findings

The central finding of this study is that **learned semantic consistency is essential for robust unsupervised segmentation in complex scenes**. While TokenCut's spectral clustering heuristics are effective for simple, single-object images (achieving 42.82% on ECSSD), they break down in realistic multi-object scenarios. EAGLE's ability to learn a consistent mapping from visual features to semantic categories allows it to overcome these limitations, handling off-center objects, multiple instances, and diverse scales with significantly higher accuracy.

However, this performance comes at the cost of training time. TokenCut remains a valuable tool for "zero-shot" applications where no training data is available, offering immediate deployment with reasonable performance on simpler tasks.

6.2 Future Work

While EAGLE establishes a strong baseline for learned unsupervised segmentation, several avenues for improvement remain. The findings of this study motivate the next phase of research focused on robustness and integration with generative models.

6.2.1 Integration with Denoising Diffusion Models

A promising direction is to integrate the discriminative segmentation capabilities of EAGLE with the generative power of Denoising Diffusion Probabilistic Models (DDPMs). Diffusion models learn rich, multi-scale representations of image structure that could complement DINO features.

- **Proposed Approach**: Use the segmentation masks from EAGLE as conditioning priors for a diffusion-based segmentation refinement step. The diffusion model could "denoise" the coarse boundaries produced by EAGLE, leveraging its learned understanding of natural image statistics to produce sharper, pixel-accurate masks.

6.2.2 Improving Robustness to Noise and Camouflage

Both methods showed performance degradation in cases of low contrast or camouflage. Future work will investigate:

- **Multi-Scale Feature Aggregation**: Combining DINO features from multiple layers to capture both high-level semantics and low-level texture details, aiding in separating camouflaged objects.
- **Contrastive Fine-Tuning**: Implementing a self-supervised fine-tuning stage specifically designed to maximize the feature distance between foreground and background in challenging scenarios.

6.2.3 Extension to Multi-Class Segmentation

This study focused on binary (thing vs. stuff) segmentation. Extending the evaluation to full multi-class segmentation on COCO-Stuff-27 would provide deeper insights.

- **Hierarchical Clustering**: Investigating hierarchical variants of EAGLE's cluster probe to automatically discover the optimal number of semantic categories, rather than assuming a fixed K.

By addressing these limitations and exploring the synergy between unsupervised segmentation and generative modeling, we aim to move closer to the ultimate goal of robust, human-level scene understanding without the need for expensive manual supervision.

REFERENCES

Primary Methods

[1] Y. Wang, X. Shen, Y. Yuan, Y. Du, M. Li, S. X. Hu, J. L. Crowley, and D. Vaufreydaz, "TokenCut: Segmenting objects in images and videos with self-supervised transformer and normalized cut," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 45, no. 12, pp. 15790–15805, Dec. 2023.

[2] C. Kim, W. Han, D. Ju, and S. J. Hwang, "EAGLE: Eigen aggregation learning for object-centric unsupervised semantic segmentation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024, pp. 3523–3532.

Backbone & Foundation

[3] M. Caron, H. Touvron, I. Misra, H. Jégou, J. Mairal, P. Bojanowski, and A. Joulin, "Emerging properties in self-supervised vision transformers," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021, pp. 9650–9660.

[4] J. Shi and J. Malik, "Normalized cuts and image segmentation," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 22, no. 8, pp. 888–905, Aug. 2000.

Datasets

[5] H. Caesar, J. Uijlings, and V. Ferrari, "COCO-Stuff: Thing and stuff classes in context," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 1209–1218.

[6] Q. Yan, L. Xu, J. Shi, and J. Jia, "Hierarchical saliency detection," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2013, pp. 1155–1162. (Reference for ECSSD)

[7] C. Wah, S. Branson, P. Welinder, P. Perona, and S. Belongie, "The Caltech-UCSD Birds-200-2011 Dataset," California Institute of Technology, Tech. Rep. CNS-TR-2011-001, 2011.

Related Unsupervised Segmentation Work

[8] M. Hamilton, Z. Zhang, B. Hariharan, N. Snavely, and W. T. Freeman, "Unsupervised semantic segmentation by distilling feature correspondences," in *International Conference on Learning Representations (ICLR)*, 2022. (Reference for STEGO, a key competitor often cited)

[9] L. Melas-Kyriazi, C. Rupprecht, I. Laina, and A. Vedaldi, "Deep spectral methods: A surprisingly strong baseline for unsupervised semantic segmentation and localization," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 8364–8375.

[10] X. Zadaianchuk, M. Kleiner, T. Brox, and V. Sitzmann, "Unsupervised semantic segmentation with self-supervised object-centric representations," in *International Conference on Learning Representations (ICLR)*, 2023.

Future Work Directions (Diffusion Models)

[11] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 6840–6851.

[12] D. Baranchuk, I. Rubachev, D. Voynov, V. Khrulkov, and A. Babenko, "Label-efficient semantic segmentation with diffusion models," in *International Conference on Learning Representations (ICLR)*, 2022.

Declaration

I/We declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, I have adequately cited and referenced the original sources. I also declare that I have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in my submission. I understand that any violation of the above will be cause for disciplinary action by the University and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

Name of the Students: Adeep Sharma, Aryaman Srivastava, Shourey Agarwal

Signatures and Date: Adeep Aryaman Shourey

28/11/2025

(for all members of the group)